

Clase 134 — Análisis dinámico y debugging de binarios

Parte: 5 — Explotación de sistemas y binarios · Fuente: Andriesse, Practical Binary Analysis 🕒

Duración estimada: 120 min · Nivel: Avanzado

Objetivo

Observar el binario **en ejecución** para revelar lo que el análisis estático no puede: valores en runtime, rutas realmente tomadas, cadenas descifradas, llamadas a librería y syscalls. Combinarás `ltrace` / `strace` , GDB con scripting, tracing con Frida y emulación selectiva, cerrando el ciclo estático↔dinámico del reversing.

⚠️ **Ética:** ejecuta binarios desconocidos solo en una VM aislada (sin red hacia producción, snapshots antes/después). Solo binarios propios/autorizados.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Trazar** llamadas a librería y syscalls con `ltrace` / `strace` .
2. **Automatizar** GDB con breakpoints condicionales y scripts.
3. **Instrumentar** funciones en runtime con Frida.
4. **Descifrar** cadenas/lógica que solo aparecen al ejecutar.
5. **Aislar** el entorno de análisis con seguridad.

Temas

#	Tema	Por qué importa
1	Aislamiento (VM/snapshots)	Ejecutar sin riesgo
2	strace (syscalls)	Qué pide al kernel
3	ltrace (funciones de librería)	Argumentos reales de <code>strcmp</code> , etc.
4	GDB scripting	Breakpoints condicionales, hooks
5	Frida	Instrumentación dinámica flexible
6	Dump de memoria	Extraer cadenas descifradas
7	Emulación (Unicorn/qiling)	Ejecutar fragmentos sin todo el entorno
8	Cierre estático↔dinámico	Metodología combinada

Definiciones y características

- **Análisis dinámico:** estudio del binario mientras corre. *Clave:* revela datos y rutas que el estático no ve.
- **strace / ltrace:** trazan syscalls y llamadas a librerías con sus argumentos. *Clave:* `ltrace` muestra el `strcmp(input, "clave")` directamente.
- **Breakpoint condicional:** se detiene solo si una condición se cumple. *Clave:* `break f if x==0x41` evita paradas inútiles.
- **Frida:** toolkit de instrumentación dinámica con scripts JS. *Clave:* hookea funciones y modifica su comportamiento en vivo.
- **Emulación selectiva (Unicorn/Qiling):** ejecutar solo una porción del binario. *Clave:* útil para desofuscar rutinas sin todo el entorno.
- **Dump de memoria:** volcar regiones para recuperar cadenas descifradas. *Clave:* en GDB con `dump memory`.

Herramientas y preparación

```
sudo apt install -y strace ltrace gdb
pip install frida-tools unicorn qiling
```

Usa una **VM aislada** con snapshot previo. Nunca ejecutes muestras desconocidas en tu host.

Laboratorio guiado

Entorno propio / VM aislada.

1. Traza syscalls y llamadas de librería del `crackme`:

```
bash strace -f ./crackme # open/read/write, etc. ltrace ./crackme # a menudo revela
strcmp(input, "SECRET")
```

2. Si `ltrace` muestra la comparación, ya tienes la clave. Si está ofuscada, sigue con GDB.

3. GDB con breakpoint condicional y hook automático:

```
gdb break strcmp commands printf "cmp: %s vs %s\n", $rdi, $rsi continue end run
```

4. Instrumenta con Frida para interceptar una función y leer sus argumentos sin recompilar:

```
javascript // hook.js Interceptor.attach(Module.getExportByName(null, "strcmp"), {
onEnter(args){ console.log("strcmp", args[0].readUtf8String(), args[1].readUtf8String()); }
});
```

```
bash frida -f ./crackme -l hook.js
```

5. Vuelca memoria para extraer una cadena descifrada en runtime (`dump memory out.bin $addr $addr+64`).

6. (Opcional) Emula una rutina de descifrado con Unicorn/Qiling para obtener la salida sin el binario completo.
7. Verifica la clave deducida ejecutando el `crackme`.

Ejercicios

1. Encuentra la clave de un `crackme` usando solo `ltrace`.
2. Escribe un script GDB que registre cada `strcmp` sin detenerse.
3. Hooke con Frida una función y modifica su valor de retorno.
4. Diferencia qué revela `strace` frente a `ltrace`.
5. Emula una función pequeña con Qiling y compara con la ejecución real.
6. Extrae una cadena descifrada de memoria con `dump memory`.

Reto verificable

Deduce la clave de un `crackme` que descifra su comparación en runtime, usando análisis dinámico (`ltrace`, GDB o Frida).

Criterio de aceptación: obtienes la clave correcta y explicas con qué herramienta/hook la capturaste en el momento de la comparación.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>ltrace</code> no muestra nada	Binario estático o anti-ltrace; usa GDB/Frida
Frida "cannot find module"	Nombre de export incorrecto; usa <code>null</code> para el binario principal
Breakpoint condicional nunca dispara	Condición mal escrita; revisa el registro
Ejecutas malware en el host	¡Riesgo! Usa VM aislada con snapshot
Cadena vacía en el dump	Rango/offset erróneo; ajusta con <code>x/s</code> antes

Preguntas frecuentes

 **¿ltrace o strace?** `ltrace` para funciones de librería (más legible); `strace` para syscalls (útil cuando no hay imports dinámicos claros).

 **¿Frida solo para móvil?** No: funciona en Linux/Windows/macOS y es excelente para desktop y CTF.

 **¿Cuándo emular?** Cuando quieres ejecutar una rutina de descifrado aislada sin montar todo el entorno del binario.

Referencias

- Andriesse, D. *Practical Binary Analysis*, cap. 9. No Starch Press.
- Frida — <https://frida.re/>
- Qiling Framework — <https://qiling.io/>

- Unicorn Engine — <https://www.unicorn-engine.org/>



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Siguiente clase

[Clase 135 - Ofuscacion y tecnicas anti-reversing](#)